

Building a Sci-Fi Themed Dense Plasma Focus Simulation Front-End in Unity

Figure 1: Concept art of a futuristic plasma simulation interface, combining volumetric plasma visuals with a neon-lit sci-fi UI.

Creating a high-fidelity **Dense Plasma Focus (DPF)** simulation front-end in Unity (v6.1 for macOS) involves setting up a Unity project, designing a stylish **neon sci-fi user interface**, and implementing advanced visualizations (particles, volumetric fields, etc.) driven by real scientific data. This step-by-step guide assumes you have Unity installed but no prior Unity development experience. We will use the Unity Editor (no command-line tools) and go through **project setup, scene configuration, UI layout, data ingestion, and interactive visualization features** in detail. By the end, you'll have a production-ready front-end that looks like a sci-fi game yet accurately displays your DPF simulation data.

Project Setup and Initialization

1. Install/Launch Unity and Create a New Project: Open Unity Hub and create a new project. Select the **3D (URP)** template for a modern render pipeline with high-fidelity graphics ¹. This ensures we have the Universal Render Pipeline and post-processing support out of the box. Name the project (e.g., "DPFSimulationViewer") and choose a location, then click **Create**. Unity will set up a new project with URP configured.

- **Unity Version:** Ensure you're using Unity 2021 or newer (Unity 6.1 corresponds to a 2024 release, which supports URP and Visual Effect Graph out-of-the-box ²). This tutorial uses URP for its balance of quality and performance.

2. Organize Project Folders: In the Unity Editor, locate the **Project** window (usually bottom pane). Right-click in the Assets folder and create subfolders to keep assets organized:

- **Scenes** – for Unity scene files.
- **Scripts** – for C# scripts.
- **Shaders** – for shader code (for volumetric rendering).
- **Materials** – for materials (volume, slice, etc.).
- **VFX** – for Visual Effect Graph assets (particle effects).
- **Textures** – for any texture assets (if needed for UI or color gradients).

This structure helps manage content as the project grows. Good organization is a best practice from the start.

3. Save the Initial Scene: Unity typically opens with a sample scene. Save it in the **Scenes** folder (e.g., File → Save As... "MainScene.unity"). This will be our main simulation viewer scene.

4. Project Settings for Rendering and Color: Since we chose a URP template, Unity has configured the Render Pipeline asset. Double-check a few settings for best results: - Go to **Edit → Project Settings → Player → Other Settings** and confirm **Color Space** is **Linear** (for correct lighting and post-processing) ³ ⁴ . - In **Project Settings → Quality**, ensure the URP pipeline asset is assigned. The URP asset will enable features like Bloom (used for neon glow). - In **Window → Rendering → Lighting**, you can disable the default skybox or set **Environment Lighting** background color to black/dark, since a neon sci-fi UI often uses dark space backgrounds for contrast.

5. Import Required Packages: We will use Unity's **Visual Effect Graph (VFX Graph)** for GPU particle rendering. In Unity Editor, open **Window → Package Manager**, set it to **Unity Registry** (All packages) ⁵ . Find **Visual Effect Graph** and click **Install** ⁵ . (If using Unity 2022+, VFX Graph may already be installed with URP ⁶ .) Unity might prompt to enable **VFX** in the URP settings; if so, accept it. This adds the VFX Graph tools to your project.

- *Verify Compute Support:* VFX Graph uses GPU compute shaders ⁷ – ensure you're on a system that supports it (most modern Macs/PCs do). On macOS, Unity will use Metal which supports compute on compatible GPUs.

Now the project is initialized with URP and VFX capabilities. Next, we'll set up the 3D scene and camera.

Scene Configuration and Neon Visual Style

1. Set Up the Main Camera: In the Hierarchy (left pane), select the **Main Camera**. In the **Inspector** (right pane): - Position the camera so it can see the origin where our simulation data will be (e.g., position = (0, 0, -5) looking toward (0,0,0)). Set **Rotation** to (0, 0, 0) to face forward along the Z-axis. - Set **Background** to solid black (click the Camera component's Background color and choose black). This gives a dark backdrop common in sci-fi visuals. - Ensure **Post Processing** is enabled on the Camera (this checkbox appears under Camera component if URP pipeline is in use). - Add a **Physics Raycaster** component to the camera (Component → Event → Physics Raycaster), which is useful if we later want the UI system to detect clicks on 3D objects (for the probe clicks).

2. Lighting: For a futuristic glowing aesthetic, we rely on emissive materials and bloom, so ambient lighting can be minimal. You can: - Delete the default Directional Light or turn it very dim (optional). We will visualize plasma as self-emissive (glowing) so direct lights aren't strictly necessary. If you keep it, set its **Intensity** low and color to a cool subtle tone so it doesn't wash out the glow. - In **Window → Rendering → Lighting → Environment**, set **Ambient Mode** to Color and choose a dark bluish color (to simulate subtle ambient light).

3. Post-Processing for Glow: To achieve the neon glow and other futuristic effects, set up a **Global Volume** for post-processing: - In Hierarchy, create an empty object named "GlobalVolume". Add **Component → Volume** and check **Is Global**. - Click **Add Override → Post-processing** and add **Bloom**. In Bloom settings, increase **Intensity** (e.g., 0.5 to 1) and lower the **Threshold** (e.g., ~1) so that bright emissive elements glow. You'll fine-tune this later while testing – the idea is to make UI highlights and plasma emissions bloom nicely. - (Optional) Add **Chromatic Aberration** (for slight color fringing) and **Vignette** (a subtle darkening at screen edges) for a cinematic sci-fi feel. Keep these effects mild to avoid distracting from data. - Add **Color Adjustments** if you want to tint the scene (e.g., a slight blue or green tint can enhance the sci-fi look).

Increasing **Contrast** can also make colors pop. - Ensure **HDR** is enabled in your URP asset (so that bright colors can bloom). URP typically has HDR on by default.

4. Scene Scale and Ground Reference: Our DPF simulation data (plasma, fields) will likely be within some region around the origin. For visual reference, you might create a thin wireframe box or grid representing the simulation domain (optional): - For example, `GameObject` → `3D Object` → `Cube`, and scale it to encompass the simulation volume. Then assign a transparent material or wireframe shader. This can act as a bounding box to help orient you. (This object can be toggled off in the final UI.) - Alternatively, add small **Axis indicators**: e.g., three arrows or colored lines along X, Y, Z for orientation. This is optional, but helps users know the coordinate system.

With the scene and camera set, let's build the user interface.

Designing the Neon Sci-Fi User Interface

Our UI will overlay on the screen to provide controls (timeline slider, toggles, info readouts) with a futuristic style. Good UX practices for sci-fi interfaces include maintaining clarity (readable text, high contrast) while adding visual flair (glowing accents, minimal text when possible) ⁸.

1. Create a UI Canvas: In Hierarchy, go to **GameObject** → **UI** → **Canvas**. Unity will add a Canvas and an **EventSystem** (for UI interaction). Set the Canvas **Render Mode** to **Screen Space - Overlay** (so it stays on screen) ⁹ ¹⁰. For sharp scaling on different resolutions, select the Canvas and find **Canvas Scaler** component; set **UI Scale Mode** to **Scale With Screen Size** (Reference Resolution e.g. 1920x1080) ¹¹. This ensures the UI layout scales for different screen sizes.

2. Add a Background Panel: To ensure UI elements are readable on bright 3D content, add a subtle background. Right-click Canvas → UI → **Panel**. This inserts a full-screen semi-transparent panel. In the Panel's **Inspector**, set its Color to nearly black with low opacity (e.g., RGBA 0,0,0,0.3) - this creates a translucent dark overlay. This panel can cover the whole screen or you can resize it to just cover a portion (like a footer bar). For a cleaner look, we'll make it a footer control strip: - With the Panel selected, in the Rect Transform, set **Anchor** to bottom stretch (hold Shift+Alt and click the bottom-stretch preset in the RectTool). Set its height to, say, 120 px, so it spans the bottom of the screen. - This panel will host our timeline slider and perhaps some buttons, giving them a consistent background.

3. Timeline Slider (Playback Control): The timeline scrubber lets the user scrub through simulation time steps. Under the Panel, create UI → **Slider**. Name it "TimelineSlider". Stretch it horizontally: in Rect Transform, anchor it to bottom stretch (same as panel) and set Left/Right offsets to 150 px (to leave room for labels/buttons at edges), and Y position to, say, 60 px from bottom (center of panel). - In **Slider** component, set **Min Value** = **0** and **Max Value** to the number of frames minus 1 (we'll set this dynamically when data is loaded). Set **Whole Numbers** = **true** (so it snaps to frame indices). - The slider includes a handle. You can style it later (for neon style, maybe change the Handle image color to cyan or green, and the Fill Area color as well). - Add a label: Right-click Slider → UI → **Text** (Legacy) or TextMeshPro if available. Name it "TimeLabel". Set its text to "Time: 0" initially. Position this text above or next to the slider to show current time/frame. Use a bold, bright font (white or neon color) and maybe a slight glow. (TextMeshPro provides outline and glow effects by default, which can be very useful for neon text). - We'll update this label via script whenever the slider changes value.

4. Data Toggle Buttons: We might want controls to toggle various data layers (particles, density volume, field overlays). For example: - Add a **Toggle** (UI → Toggle) to the Panel, align it to bottom-left. Label it "Particles". This will allow the user to show/hide particle rendering. - Duplicate the Toggle for "Volume", "Slice Overlay", etc., as needed. Arrange these toggles on the left side of the panel. - Style: for a sci-fi feel, you can remove the default checkmark box background and use only text or a custom icon. But as a beginner, simply using the default Toggle is fine. Change the text color to a bright neon (cyan or lime) and perhaps increase font size for visibility.

5. Probe Data Readout: We'll implement **clickable 3D probes** that when clicked, display field values at that location. We need a UI element to show this information: - Create a UI **Text** element (or TMP text) and anchor it to the top-right of the Canvas (for example). Set its text to something like "Probe: (click a probe)" initially. - Style the text: bright color (yellow or cyan) so it stands out, and maybe add a Shadow component (Component → UI → Effects → Shadow) for readability. Keep text small enough to not clutter the screen. - This text will be updated in runtime to show values (density, etc.) when a probe object in the scene is clicked.

Figure 2: Example of a scientific 3D visualization interface (ParaView/USim) with a timeline slider at the bottom and data controls on the side ¹².

UI Design Best Practices: Ensure the UI elements are **logically grouped and labeled**. For instance, keep all toggles for data layers together, and the timeline slider centered for easy access. Use consistent colors for interactive elements (e.g., all toggles and slider in neon cyan) and avoid too much text. Sci-fi interfaces often use icons or minimal text ⁸, but since we are focusing on functionality, clear labels are fine. Just keep the font and colors consistent for a cohesive look.

Now the UI exists in the scene. Next, we'll implement **data visualization components** – particle rendering and volumetric field rendering – and then wire them up with data and UI controls.

Particle Rendering with VFX Graph (GPU Instancing)

For simulating plasma particles or discrete points (if your DPF simulation has them), we use Unity's **Visual Effect Graph**. The VFX Graph is a node-based tool that runs particle simulations on the GPU, allowing us to draw tens of thousands of particles efficiently ⁷. We will configure a VFX Graph to take in our simulation's particle positions and render them as glowing points.

1. Create a Visual Effect Graph Asset: In Project window, go to **Assets/VFX** folder. Right-click → Create → **Visual Effects → Visual Effect Graph**. Name it "ParticlePoints.vfx". This opens the Visual Effect Graph Editor. (If prompted about VisualEffect not supported in this pipeline, make sure you installed it and URP supports it; we addressed this earlier.)

2. Set up Particle Capacity: In the VFX Graph Inspector (Blackboard), you'll see an **Initialize** context with a default capacity (often 256 or 1000). Since our simulation might produce many particles, set **Capacity** to a safe high number – e.g., 50,000 – so it can handle up to that many particles without culling. (You can adjust based on expected particle count.)

3. Configure Particle Spawn and Lifetime: Delete the default spawn block (if any spawning constantly). We will control spawning manually via events. Do the following: - In the Graph, add a **Single Burst** block in the Initialize context. Set the **Count** to an exposed parameter (we'll create one). - In the Blackboard (graph properties panel), click + to add a **Property** of type **UInt** and name it **ParticleCount**. Check **Exposed** (so we can set it from script). - Drag **ParticleCount** into the graph and connect it to the Count of the Single Burst. This way, whenever we trigger a burst spawn, it will spawn exactly ParticleCount particles. - Add a **Spawn Event** context if not present: Right-click in graph, Create Block → **Spawn System**. We want an event-based spawn. By default there's a "On Play" event. We can use that, or create a custom event. Let's use a custom one for clarity: In the Spawn context, click the plus icon next to **Name** and name the event "SpawnParticles". This means our graph will listen for an event called "SpawnParticles" and then execute the Single Burst to spawn particles. - Result: when we send the "SpawnParticles" event from code (with ParticleCount set), it will spawn that many particles at once. Each particle will live indefinitely unless we specify a lifetime. For static frames, we can let them live and simply respawn on frame change (clearing old ones by restarting the system). Set **Lifetime** long (or leave default 5 sec and enable **Looping** so it continuously respawns – but looping might conflict with manual spawn, so prefer very long lifetime or no expiration). For simplicity, set **Lifetime** to something like 10000 and **Looping** unchecked.

4. Set Particle Position from Data: We need to feed in the positions from our simulation. VFX Graph allows sampling from a texture or buffer to set particle attributes. We will use a Texture2D as a container for positions (where each pixel encodes a particle's position in XYZ). This is a common technique for GPU instancing data. - In the Blackboard, add a **Texture2D** property, name it **PositionMap**, and mark Exposed. Set **Texture type** to Default and **Format** to RGBA32 or RGBAFloat (we will use RGBAFloat for high precision). - In the Initialize context, add a **Set Position (Attribute from Map)** block ¹³ ¹⁴. Configure it as follows: Attribute = **position**, Sample Mode = **Sequential** (so each particle gets a unique pixel from the texture sequentially based on particleID ¹⁵ ¹⁶). Assign **PositionMap** to the Texture input of this block. - This tells VFX: for particle with ID i, sample the PositionMap at pixel [i] (sequentially) and use that RGBA value as the particle's position. We will ensure our texture's pixels [0..N-1] contain the XYZ of N particles. - If needed, also set **Space** to World (so the positions are treated as world coordinates). Alternatively, you can spawn them relative to an origin and not transform, but world space is fine here because our data will be in some world coordinate system.

5. Add Particle Appearance: By default, particles might be rendered as small quad sprites. You can customize the output: - In the **Output** context (likely a "Output Particle Quad" by default), set the **Material** to an unlit additive material if not already. The default VFX Particle Quad uses a simple particle shader that's additive (good for glow). You can also set a **Color** over life or a constant color. For example, add a **Set Color** block in Initialize or Output and choose a bright cyan or white. Because we'll rely on bloom, white points will glow in whatever bloom tint we set. Alternatively, a cyan color with bloom will glow cyan. - Set **Size**: add a **Set Size** block. You might start with a small size like 0.02 (relative to world units). We can also expose this as a parameter if desired. Keep them small so they look like points, but not so small they vanish. - You might want **Depth Sorting** enabled (in Output context settings, check "Sort" so nearer particles render in front of farther ones, if you have translucency/additive overlap issues). - With these settings, the VFX Graph is ready. Save the graph (Ctrl+S).

6. Place VFX in the Scene: In your Hierarchy, create an empty GameObject named "ParticleEffect". Add **Visual Effect** component to it (Component → Visual Effects → Visual Effect). In the Visual Effect component, assign the **ParticlePoints.vfx** asset to it. You should see in the Inspector the exposed properties (PositionMap and ParticleCount) listed, and possibly a default bounding box gizmo. - Position this

GameObject at the origin (0,0,0). This will be the anchor for particles if you use local space. We set positions in world space, so it can actually be anywhere, but keep it at origin for consistency. - Initially, nothing will spawn until we trigger our event via script.

We will script the logic to send data to this Visual Effect. The idea is: - Convert our particle positions (from simulation frame) into a Texture2D (PositionMap). - Set that texture and count on the VisualEffect. - Send the "SpawnParticles" event to refresh particles each frame change.

We'll implement this in the main controller script soon. But first, let's also set up the volumetric field visualization.

Volumetric Density Field Rendering (Shader-based Raymarching)

A key part of our front-end is visualizing volumetric field data, such as plasma density throughout the simulation volume. We will use a **raymarching shader** to render a 3D density field as a glowing volume. Raymarching means the shader will cast rays through a 3D texture (volume texture) and accumulate color/opacity based on density ¹⁷ ¹⁸. This produces a cloud-like or foggy volumetric effect where dense regions appear bright/opaque.

1. Create a Volume Shader: In the **Shaders** folder, right-click → Create → **Shader** → **Unlit Shader**. Name it "VolumeRaymarch.shader". Open it in the code editor. Replace its content with the following shader code (this is adapted from Unity's documentation example ¹⁹ ²⁰, with modifications for our use-case):

```
Shader "Unlit/VolumeRaymarch" {
    Properties {
        _MainTex ("Density Volume", 3D) = "" {} // 3D volume texture
        _ColorTint ("Color Tint", Color) = (1,1,1,1) // Tint color for the
        volume
        _Alpha ("Density Alpha Scale", Float) = 0.04 // Opacity scale for
        density
        _StepSize ("Raymarch Step Size", Float) = 0.01 // Step length in volume
        sampling
    }
    SubShader {
        Tags { "Queue"="Transparent" "RenderType"="Transparent" }
        Blend SrcAlpha OneMinusSrcAlpha // standard alpha blending
        Pass {
            CGPROGRAM
            #pragma vertex vert
            #pragma fragment frag
            #include "UnityCG.cginc"
            // Max number of samples along ray
            #define MAX_STEP_COUNT 256
            #define EPSILON 1e-5

            struct appdata { float4 vertex : POSITION; };
```

```

struct v2f {
    float4 pos : SV_POSITION;
    float3 localPos : TEXCOORD0;
    float3 camToVertex : TEXCOORD1;
};

sampler3D _MainTex;
float4 _MainTex_ST;           // not used but Unity defines it
float4 _ColorTint;
float _Alpha;
float _StepSize;

v2f vert (appdata v) {
    v2f o;
    // vertex in object (local) space:
    o.localPos = v.vertex.xyz;
    // world position of vertex:
    float3 worldPos = mul(unity_ObjectToWorld, v.vertex).xyz;
    // vector from camera to worldPos, then to object space:
    o.camToVertex = mul(unity_WorldToObject,
float4(_WorldSpaceCameraPos,1)).xyz - o.localPos;
    o.pos = UnityObjectToClipPos(v.vertex);
    return o;
}

fixed4 frag(v2f i) : SV_Target {
    // Starting point (entry) in object space:
    float3 rayOrigin = i.localPos;
    // Ray direction from camera into the object (object space):
    float3 rayDir = normalize(i.camToVertex);
    fixed4 accumulatedColor = 0;
    float3 samplePos = rayOrigin;

    // Ray-march through the volume (object's local bounds -0.5 to
0.5)
    [loop] for (int step = 0; step < MAX_STEP_COUNT; step++) {
        if (max(abs(samplePos.x), max(abs(samplePos.y),
abs(samplePos.z))) > 0.5f) {
            break; // exit if we're outside the cube
        }
        // Sample density texture (shift from [-0.5,0.5] to [0,1]
UVW)
        float density = tex3D(_MainTex, samplePos + 0.5f).a;
        // We use the texture's alpha as density (assuming density
stored in alpha)
        if (density > EPSILON) {
            // Compute color contribution
            fixed4 voxelColor = _ColorTint * density;

```

```

        voxelColor.a = density * _Alpha;
        // BlendUnder: composite current voxelColor under
        accumulatedColor
        accumulatedColor.rgb += (1 - accumulatedColor.a) *
        voxelColor.a * voxelColor.rgb;
        accumulatedColor.a += (1 - accumulatedColor.a) *
        voxelColor.a;
        // If fully opaque, we can break (for performance,
        optional)
        if(accumulatedColor.a >= 0.99f) { break; }
    }
    samplePos += rayDir * _StepSize;
}
return accumulatedColor;
}
ENDCG
}
}
}
}

```

Let's unpack this shader: - It defines a 3D texture `_MainTex` which will hold our density volume. - In the vertex shader (`vert`), it computes the position of vertices in object-local space and the direction from camera through the object ²¹ ²². - The fragment shader (`frag`) then marches along that ray in small steps (`_StepSize`) through the cube. It samples the 3D texture at each step ²³. We assume the density value is stored in the *alpha channel* of the texture (`tex3D(...).a`). We then accumulate color and alpha: essentially doing volumetric compositing (front-to-back alpha blending) ²³ ²⁰. - `_ColorTint` allows tinting the volume's color (e.g., a blue or orange glow). `_Alpha` scales the density's opacity influence ²⁴. - The loop breaks if the ray exits the cube bounds or if accumulated alpha ~1 (fully opaque). - The shader tags make it render in the Transparent queue with alpha blending.

2. Create Material and Volume Object: Save the shader. Now create a **Material** using this shader: - Project window → Materials folder, right-click Create → **Material**, name it "VolumeMat". In Inspector, select **Shader** = **Unlit/VolumeRaymarch** (our custom shader). - Choose a `_ColorTint` for the plasma, e.g., cyan (0,1,1) or purple (1,0,1) with alpha 1. This is the base color of the glow. Leave `_Alpha` at 0.04 as defined (you can tweak how opaque the density appears) and `_StepSize` at 0.01 (smaller step = higher quality but slower). - Now create the volume object in the scene: GameObject → 3D Object → **Cube**. Name it "DensityVolume". - This cube will be our volume container. By default, Unity's cube mesh is 1 unit size (extents from -0.5 to +0.5 in local space), which matches our shader's expected volume bounds. **Do not scale the cube**; keep its Scale at (1,1,1) for the shader math to remain correct. (If you need the volume to cover a different physical size, we will map data accordingly rather than scaling the cube, to avoid complicating the raymarching.) - Remove the Box Collider component (not needed for visualization). - Assign **VolumeMat** to the cube's MeshRenderer. The cube will disappear or look invisible now, because currently `_MainTex` is empty and our shader likely finds zero density. That's fine. - Position the cube so it covers the region of interest. If your simulation domain is roughly cubic and centered at origin, place the cube at (0,0,0). If the domain is rectangular, you could stretch the cube non-uniformly to fit, *but* note that our shader assumes a unit cube sampling. For simplicity, we assume a roughly equal aspect domain and keep a unit cube. (Advanced: one

could scale the cube and adjust sampling directions accordingly, but that's beyond our scope.) - The cube with VolumeMat will render the density once we supply a 3D texture.

3. Prepare 3D Texture for Density: Unity requires a **Texture3D** asset or dynamically created Texture3D for volume data. We will create it at runtime from our simulation data (so no need to import a .asset). The shader expects the density in the texture's alpha channel. We will fill the texture such that `alpha = densityNormalized` and perhaps set RGB=1 for tinted color (or leave RGB white since we multiply by `_ColorTint` in shader). - We will do this in code: create `Texture3D densTex = new Texture3D(nx, ny, nz, TextureFormat.RFloat, false)` or `RGBAFloat` (RFloat might suffice if we only use alpha). To keep it simple, we'll use RGBAFloat and put density in alpha. - Our script will then assign `densTex` to the VolumeMat's `_MainTex` property, and the shader will then render the volume. - We will also handle updating this texture each frame or when new data comes in.

We now have the volumetric rendering infrastructure: a shader and an object to display it. Next, we consider **field overlays** like slice planes.

2D Field Slice Overlays (Optional)

Many scientific visualizations show cross-sections or contour slices of field data. We can overlay a 2D colored slice in the 3D scene. For example, we might show a cut-plane of density or temperature. To demonstrate, we'll add one slice plane for density:

1. Create a Slice Plane: GameObject → 3D Object → **Quad**. Name it "DensitySlice". This is a flat 1x1 plane (XY plane by default) with its front face pointing forward (positive Z). We need to orient it to slice through our volume. For example, to create an X-Y plane slicing at mid Z: - Rotate the Quad 90° around X-axis to make it lie flat in XZ or rotate around Y if needed. Actually, Unity's Quad lies in the XY plane by default; to make it a Y-Z plane (slice along X axis), rotate 90° around Y. To make an X-Y horizontal slice, rotate 90° around X. Decide which orientation corresponds to the slice you want. - For instance, let's do a horizontal (XY) slice at mid-height of the volume. Rotate Quad by **90° around X-axis** so it lies horizontally (so its surface is parallel to the ground). Position it at Y=0 (mid of volume if volume spans roughly -0.5 to 0.5 in Y). - Scale the Quad to match the volume's X and Z dimensions. Since our volume cube is 1x1x1, scale the Quad to 1 unit in X and 1 unit in Z (Y doesn't matter as it's flat). So scale (1,1,1) already covers a 1x1 plane which exactly covers the top face of the cube. If needed, adjust scale if your volume is scaled differently. - Ensure the Quad is slightly above the volume to avoid Z-fighting (since it might coincide with the cube's face). For example, position Y = 0.001 or so if slicing at Y=0, or if a vertical slice, offset a tiny bit along that axis.

2. Create a Slice Material: In **Materials**, create a new Material "SliceMat". Use shader **Unlit/Transparent** (or Unlit/Unlit Texture if using built-in pipeline; in URP, you can use Unlit shader graph, but simplest is built-in unlit for now). We want to apply a texture that represents density values on this plane. - Set Material color tint to white with some alpha (e.g., RGBA 1,1,1,0.5) for semi-transparency. - Assign "SliceMat" to the Quad's Renderer.

3. Prepare Slice Texture: We will generate a **Texture2D** at runtime for the slice. The texture size should match the data grid on that slice. E.g., if our volume grid is nx×ny×nz, and we're slicing at a given index, the slice might be an `nx×nz` image (for a horizontal slice at certain Y) or `nx×ny` if vertical slice at certain Z, etc. We'll decide: - Suppose we do a horizontal slice at mid-Z (parallel to X-Y plane). That texture should be

`nx × ny` (covering the full X-Y cross-section). - If doing a vertical slice (Y-Z), texture would be `ny × nz`, etc. In code we can extract whichever slice we want. - For demonstration, let's say a horizontal mid-plane: index `k = nz/2`. We'll create an `nx × ny` Texture2D and fill it with density values from `density[:, :, k]` (i.e., all *i,j* at that fixed *k*). - We can map density to a color map. For simplicity, we can do grayscale or a single-color gradient. Let's do a blue color map: low density = black transparent, high density = bright cyan. Or use Unity's built-in color gradient if we had, but manual is fine. - The script will compute min/max or use global max to normalize. Then color = (0, norm, norm, 1) perhaps to get black->cyan (since R=0, G=B=norm). - We set the texture and assign it to SliceMat's MainTexture.

The slice will then overlay on the volume. Because the slice material is transparent, it can be seen along with the volume. You might need to adjust rendering order: - By default, our volume shader is Transparent queue (around 3000). The Unlit/Transparent material also is in Transparent queue. The rendering order might be undefined if both have same SortingLayer/Order. If the slice sometimes appears behind the volume when it shouldn't (or vice versa), we can tweak: - E.g., set the SliceMat's render queue explicitly to 3100 (click material, in Inspector, under "Custom Render Queue" type 3100) to ensure it renders after the volume. This way the slice will be drawn on top of the volume. This is useful if the slice should always be visible. Alternatively, we could disable depthWrite on the slice material (but not needed if we just draw it last). - We will also allow the user to toggle this slice on/off via the UI toggle we created.

At this stage, we have: - **ParticleEffect** (VisualEffect for particles). - **DensityVolume** (cube with volumetric shader). - **DensitySlice** (quad with 2D field texture). - UI elements ready.

The remaining step is to **write the scripts** that load scientific data and link everything together interactively.

Data Ingestion and Linking Real Scientific Data

We need to handle two data sources: - **Static local files** (e.g., JSON metadata + binary dumps) for simulation outputs saved on disk. - **Live data via WebSocket** for real-time updates (e.g., simulation running and streaming frames).

We will design a **SimulationController** script that on startup can load static data from file, connect to a WebSocket if needed, and manage updating the visuals when new data frames are available. This script will also handle UI events (slider change, toggles, etc.).

Let's outline the data structures: - A `FrameData` class to hold one frame's data: particle positions array, density field array, etc. We can also include time or step index if needed. - A list `frames` to store all loaded FrameData (for static data). - For live data, we might not store all frames (if streaming continuously, we could display latest or accumulate limited history). But we can also accumulate into frames list if we want the slider to access previous time steps. (This depends on memory and use-case.) - Queues for incoming data from WebSocket (to safely transfer data from network thread to main thread). - References to Unity objects: VisualEffect, materials, UI elements, etc., to update them.

We will write the `SimulationController` as a Unity MonoBehaviour (attach it to an empty GameObject "SimulationController" in scene). It will manage everything.

1. Define FrameData class (C#): In **Scripts** folder, create a new C# script **FrameData.cs** (or we can define it inside **SimulationController** script for simplicity, using an inner class or just define a struct). For clarity, a separate class:

```
public class FrameData {
    public Vector3[] particles;
    public float[, ,] density;    // 3D array [nx, ny, nz] of density
    // You could add other fields: e.g., float[, ,] temperature, etc.
    public int frameIndex;
    public float time;    // simulation time for this frame (if available)
}
```

We use a 3D array for density for easy indexing. Alternatively, could use a 1D array of length $n_x n_y n_z$, but 3D is more intuitive for sampling by index.

2. SimulationController Script: Create **SimulationController.cs** in **Scripts** folder and open it. We'll implement the following features: - Public fields to link Unity objects (VisualEffect, materials, UI components, etc.). - Methods to load data from file, to connect to WebSocket, and to update visuals when frame changes. - Event handlers for UI (slider change, toggle change). - Methods to sample data for probes.

Below is a comprehensive implementation:

```
using UnityEngine;
using UnityEngine.UI;
using UnityEngine.VFX;
using System;
using System.IO;
using System.Text;
using System.Collections.Generic;
using System.Net.WebSockets;
using System.Threading;
using System.Threading.Tasks;

public class SimulationController : MonoBehaviour {
    [Header("Unity Scene References")]
    public VisualEffect particleEffect;    // Drag the ParticleEffect
    VisualEffect object here
    public Material volumeMaterial;    // Drag the VolumeMat material here
    public Material sliceMaterial;    // Drag the SliceMat material here
    (for field overlay)
    public Slider timelineSlider;    // Drag the TimelineSlider UI here
    public Text timeLabel;    // Drag the TimeLabel UI Text here
    public Toggle particlesToggle;    // Toggle for particles on/off
    public Toggle volumeToggle;    // Toggle for volume on/off
    public Toggle sliceToggle;    // Toggle for slice overlay on/off
}
```

```

    public Text probeInfoText;           // Text UI for probe readout

    [Header("Data Settings")]
    public TextAsset jsonDataFile;       // (Optional) Drag a JSON file from
Assets if included, or set path below
    public string binaryDataPath;       // Path to binary data file on disk
(streaming assets or absolute)
    public string websocketURL = "ws://localhost:12345"; // WebSocket URL for
live data

    // Internal data structures
    private List<FrameData> frames = new List<FrameData>();
    private int currentFrame = 0;
    private int nx, ny, nz; // grid dimensions
    private ClientWebSocket ws;
    private Thread wsThread;
    private Queue<string> wsJsonQueue = new Queue<string>(); // queue for
incoming JSON messages
    private Queue<byte[]> wsBinaryQueue = new Queue<byte[]>(); // queue for
incoming binary data

    void Start() {
        // Link UI events
        timelineSlider.onValueChanged.AddListener(OnTimelineSliderChanged);
        if (particlesToggle)
particlesToggle.onValueChanged.AddListener(OnParticlesToggle);
        if (volumeToggle)
volumeToggle.onValueChanged.AddListener(OnVolumeToggle);
        if (sliceToggle) sliceToggle.onValueChanged.AddListener(OnSliceToggle);

        // Initialize scene object states
        UpdateVisibility(); // apply initial toggle states
        probeInfoText.text = ""; // start with empty probe info

        // Load static data if available
        if (jsonDataFile != null) {
            Debug.Log("Loading simulation data from JSON and binary...");
            LoadSimulationFromFiles(jsonDataFile, binaryDataPath);
        }

        // Set up WebSocket connection (if URL provided)
        if (!string.IsNullOrEmpty(websocketURL)) {
            StartWebSocketListener(websocketURL);
        }

        // If we loaded frames from file, initialize first frame visuals
        if (frames.Count > 0) {
            currentFrame = 0;

```

```

        ApplyFrameData(currentFrame);
        timelineSlider.minValue = 0;
        timelineSlider.maxValue = frames.Count - 1;
        timelineSlider.value = 0;
        UpdateTimeLabel();
    }
}

void Update() {
    // Handle incoming WebSocket data on main thread
    lock(wsJsonQueue) {
        while (wsJsonQueue.Count > 0) {
            string json = wsJsonQueue.Dequeue();
            ProcessIncomingJson(json);
        }
    }
    lock(wsBinaryQueue) {
        while (wsBinaryQueue.Count > 0) {
            byte[] data = wsBinaryQueue.Dequeue();
            ProcessIncomingBinary(data);
        }
    }
}

// --- Data Loading Methods ---
void LoadSimulationFromFiles(TextAsset jsonTextAsset, string binaryPath) {
    // Parse JSON meta-data
    var jsonText = jsonTextAsset.text;
    SimulationMeta meta = JsonUtility.FromJson<SimulationMeta>(jsonText);
    // Assuming SimulationMeta is a class that matches JSON structure with
    fields like:
    // public int frameCount; public int nx, ny, nz; public int
    particleCount; etc.
    nx = meta.nx;
    ny = meta.ny;
    nz = meta.nz;
    int nFrames = meta.frameCount;
    int particleCount = meta.particleCount;
    Debug.Log($"Simulation grid: {nx}x{ny}x{nz}, frames: {nFrames},
    particles per frame: {particleCount}");

    // Open binary file
    using (BinaryReader br = new BinaryReader(File.Open(binaryPath,
    FileMode.Open))) {
        for (int f = 0; f < nFrames; f++) {
            FrameData frame = new FrameData();
            frame.frameIndex = f;
            frame.time = meta.times != null && meta.times.Length > f ?

```

```

meta.times[f] : f;
    // Read particles
    frame.particles = new Vector3[particleCount];
    for (int p = 0; p < particleCount; p++) {
        float px = br.ReadSingle();
        float py = br.ReadSingle();
        float pz = br.ReadSingle();
        frame.particles[p] = new Vector3(px, py, pz);
    }
    // Read density grid
    frame.density = new float[nx, ny, nz];
    for (int k = 0; k < nz; k++) {
        for (int j = 0; j < ny; j++) {
            for (int i = 0; i < nx; i++) {
                frame.density[i, j, k] = br.ReadSingle();
            }
        }
    }
    frames.Add(frame);
}
}

void StartWebSocketListener(string url) {
    ws = new ClientWebSocket();
    wsThread = new Thread(() => {
        try {
            ws.ConnectAsync(new Uri(url), CancellationToken.None).Wait();
            Debug.Log("WebSocket connected!");
            var buffer = new byte[1024 * 1024 * 4]; // 4 MB buffer for
messages
            while (ws.State == WebSocketState.Open) {
                var result = ws.ReceiveAsync(new
ArraySegment<byte>(buffer), CancellationToken.None).Result;
                if (result.MessageType == WebSocketMessageType.Close) {
                    Debug.Log("WebSocket closed by server.");
                    break;
                }
                if (result.MessageType == WebSocketMessageType.Text) {
                    string msg = Encoding.UTF8.GetString(buffer, 0,
result.Count);
                    lock(wsJsonQueue) { wsJsonQueue.Enqueue(msg); }
                }
                else if (result.MessageType == WebSocketMessageType.Binary)
{
                    // copy exact length data
                    byte[] dataCopy = new byte[result.Count];
                    Array.Copy(buffer, dataCopy, result.Count);

```

```

        lock(wsBinaryQueue) { wsBinaryQueue.Enqueue(dataCopy); }
    }
}
} catch (Exception e) {
    Debug.LogError("WebSocket error: " + e.Message);
}
});
wsThread.IsBackground = true;
wsThread.Start();
}

// Process JSON messages from WebSocket (e.g., metadata or frame info)
void ProcessIncomingJson(string json) {
    // Here we parse incoming JSON. It could contain frame metadata or small
data.
    // For example, maybe it contains "frameIndex" or "densitySlice" etc.
    // This will depend on your WebSocket protocol. As a simple example:
    try {
        LiveFrameMessage msg = JsonUtility.FromJson<LiveFrameMessage>(json);
        if (msg.frameIndex >= 0 && msg.totalFrames > 0) {
            // If first message gives sim dimensions, allocate frames list
            if (frames.Count == 0 && msg.totalFrames > 0) {
                nx = msg.nx; ny = msg.ny; nz = msg.nz;
                frames = new List<FrameData>(new
FrameData[msg.totalFrames]);
            }
            // Possibly the JSON indicates next binary message content (like
"next message will be density for frame X")
            // We'll handle binary in ProcessIncomingBinary.
        }
    } catch (Exception e) {
        Debug.LogWarning("Failed to parse incoming JSON: " + e.Message);
    }
}

// Process binary data from WebSocket (e.g., frame arrays)
void ProcessIncomingBinary(byte[] data) {
    // Interpret binary data according to known format.
    // Let's assume for example: first 4 bytes = int frameIndex, next 4 =
int particleCount,
    // then particleCount*3*4 bytes of floats for positions, then nx*ny*nz*4
bytes for density floats.
    using (BinaryReader br = new BinaryReader(new MemoryStream(data))) {
        int fIndex = br.ReadInt32();
        int pCount = br.ReadInt32();
        // Ensure frames list can hold this index
        if (frames.Count <= fIndex) {
            frames.AddRange(new FrameData[fIndex - frames.Count + 1]);

```

```

    }
    FrameData frame = new FrameData();
    frame.frameIndex = fIndex;
    frame.particles = new Vector3[pCount];
    for (int p = 0; p < pCount; p++) {
        float px = br.ReadSingle();
        float py = br.ReadSingle();
        float pz = br.ReadSingle();
        frame.particles[p] = new Vector3(px, py, pz);
    }
    frame.density = new float[nx, ny, nz];
    for (int k = 0; k < nz; k++) {
        for (int j = 0; j < ny; j++) {
            for (int i = 0; i < nx; i++) {
                if (br.BaseStream.Position < br.BaseStream.Length) {
                    frame.density[i,j,k] = br.ReadSingle();
                }
            }
        }
    }
    frames[fIndex] = frame;
    Debug.Log($"Received frame {fIndex} via WebSocket");
    // If this is the first frame received, initialize the slider and
visuals
    if (fIndex == 0) {
        timelineSlider.minValue = 0;
        timelineSlider.maxValue = Mathf.Max(frames.Count-1, fIndex);
        currentFrame = 0;
        ApplyFrameData(0);
        UpdateTimeLabel();
    }
    // If live streaming, we might automatically advance to new frame:
    if (fIndex > currentFrame) {
        currentFrame = fIndex;
        ApplyFrameData(currentFrame);
        // Update slider without triggering event to avoid double apply:
        timelineSlider.SetValueWithoutNotify(currentFrame);
        UpdateTimeLabel();
    }
}

// --- Visualization Update Methods ---
void ApplyFrameData(int frameIndex) {
    if (frameIndex < 0 || frameIndex >= frames.Count || frames[frameIndex]
== null) return;
    FrameData frame = frames[frameIndex];
    // Update Particles

```

```

    UpdateParticleEffect(frame.particles);
    // Update Volume 3D texture
    UpdateDensityVolume(frame.density);
    // Update Slice texture (example: horizontal mid-slice at k = nz/2)
    int sliceK = nz/2;
    UpdateSliceTexture(frame.density, sliceK);
}

void UpdateParticleEffect(Vector3[] particles) {
    if (particleEffect == null) return;
    int count = particles.Length;
    // Create or reuse a PositionMap texture with width = particle count
    Texture2D posTex = new Texture2D(count, 1, TextureFormat.RGBAFloat,
false);
    Color[] cols = new Color[count];
    for (int i = 0; i < count; i++) {
        Vector3 p = particles[i];
        // If simulation coordinates are not in Unity world units, you might
need to scale or offset here.
        cols[i] = new Color(p.x, p.y, p.z, 1.0f);
    }
    posTex.SetPixels(cols);
    posTex.Apply();
    // Assign to visual effect
    particleEffect.SetTexture("PositionMap", posTex);
    particleEffect.SetUInt("ParticleCount", (uint)count);
    particleEffect.SendEvent("SpawnParticles");
}

void UpdateDensityVolume(float[,,] densityGrid) {
    if (volumeMaterial == null) return;
    int sx = densityGrid.GetLength(0);
    int sy = densityGrid.GetLength(1);
    int sz = densityGrid.GetLength(2);
    // Create 3D texture
    Texture3D tex3D = new Texture3D(sx, sy, sz, TextureFormat.RGBAFloat,
false);
    Color[] voxels = new Color[sx * sy * sz];
    // We will normalize density to [0,1] for visualization. For simplicity,
use global max of this frame.
    float maxVal = 0f;
    foreach (float val in densityGrid) { if (val > maxVal) maxVal = val; }
    if (maxVal == 0) maxVal = 1; // avoid divide by zero
    int idx = 0;
    for (int k = 0; k < sz; k++) {
        for (int j = 0; j < sy; j++) {
            for (int i = 0; i < sx; i++) {
                float dens = densityGrid[i,j,k] / maxVal;

```

```

        // Clamp dens to [0,1]
        if (dens < 0) dens = 0;
        if (dens > 1) dens = 1;
        // Store density in alpha, white color in RGB (or you could
leave RGB 0 since shader uses _ColorTint)
        voxels[idx++] = new Color(1f, 1f, 1f, dens);
    }
}
tex3D.SetPixels(voxels);
tex3D.Apply();
volumeMaterial.SetTexture("_MainTex", tex3D);
}

void UpdateSliceTexture(float[,,] densityGrid, int sliceIndex) {
    if (sliceMaterial == null) return;
    int sx = densityGrid.GetLength(0);
    int sy = densityGrid.GetLength(1);
    int sz = densityGrid.GetLength(2);
    if (sliceIndex < 0 || sliceIndex >= sz) sliceIndex = sz/2;
    // Create Texture2D for this slice (size: sx x sy)
    Texture2D sliceTex = new Texture2D(sx, sy, TextureFormat.RGBA32, false);
    // We will find min/max on this slice for normalization
    float minVal = float.MaxValue;
    float maxVal = float.MinValue;
    for (int j = 0; j < sy; j++) {
        for (int i = 0; i < sx; i++) {
            float val = densityGrid[i, j, sliceIndex];
            if (val < minVal) minVal = val;
            if (val > maxVal) maxVal = val;
        }
    }
    if (maxVal <= minVal) maxVal = minVal + 1e-5f;
    Color[] pixels = new Color[sx * sy];
    for (int j = 0; j < sy; j++) {
        for (int i = 0; i < sx; i++) {
            float val = densityGrid[i, j, sliceIndex];
            float norm = (val - minVal) / (maxVal - minVal);
            // Map to color (here: black -> blue -> cyan for low->high)
            float r = 0f;
            float g = norm;
            float b = 0.5f + 0.5f * norm; // goes from 0.5 (dark blue) to 1
(cyan)

            // We set alpha to a constant transparency
            pixels[j * sx + i] = new Color(r, g, b, 0.7f * norm);
        }
    }
    sliceTex.SetPixels(pixels);
}

```

```

        sliceTex.Apply();
        sliceMaterial.SetTexture("_MainTex", sliceTex);
    }

    // --- UI Event Handlers ---
    public void OnTimelineSliderChanged(float newVal) {
        int frameIndex = Mathf.RoundToInt(newVal);
        if (frameIndex != currentFrame) {
            currentFrame = frameIndex;
            ApplyFrameData(currentFrame);
            UpdateTimeLabel();
        }
    }

    void UpdateTimeLabel() {
        if (frames.Count > 0 && currentFrame < frames.Count &&
frames[currentFrame] != null) {
            float t = frames[currentFrame].time;
            timeLabel.text = $"Time: {t:0.00}"; // assuming time in
microseconds or such, format as needed
        } else {
            timeLabel.text = $"Frame: {currentFrame}";
        }
    }

    void OnParticlesToggle(bool enabled) {
        if (particleEffect != null) {
            particleEffect.gameObject.SetActive(enabled);
        }
    }

    void OnVolumeToggle(bool enabled) {
        if (volumeMaterial != null) {
            // Toggle rendering the volume: easiest is to enable/disable the
whole DensityVolume object
            GameObject volObj = GameObject.Find("DensityVolume");
            if (volObj) volObj.SetActive(enabled);
        }
    }

    void OnSliceToggle(bool enabled) {
        GameObject sliceObj = GameObject.Find("DensitySlice");
        if (sliceObj) sliceObj.SetActive(enabled);
    }

    void UpdateVisibility() {
        // Apply initial toggle states to objects:
        if (particlesToggle) OnParticlesToggle(particlesToggle.isOn);
        if (volumeToggle) OnVolumeToggle(volumeToggle.isOn);
        if (sliceToggle) OnSliceToggle(sliceToggle.isOn);
    }

```

```

// --- Probe sampling for clickable probes ---
public float SampleDensityAt(Vector3 worldPos) {
    if (frames.Count == 0 || frames[currentFrame] == null) return 0f;
    // Assuming simulation coordinates align with Unity world (if not,
transform worldPos to sim grid coords here)
    FrameData frame = frames[currentFrame];
    // Map world position to grid indices
    // Here assume volume cube covers range [-0.5,0.5] in each axis (as per
our volume shader).
    // If simulation domain has real extents (e.g., length Lx, etc.), we
should map accordingly.
    // For simplicity, assume simulation is normalized to [-0.5,0.5] as
well.

    float localX = worldPos.x; // (if needed, subtract origin)
    float localY = worldPos.y;
    float localZ = worldPos.z;
    // Convert to index
    int i = Mathf.FloorToInt((localX + 0.5f) * nx);
    int j = Mathf.FloorToInt((localY + 0.5f) * ny);
    int k = Mathf.FloorToInt((localZ + 0.5f) * nz);
    i = Mathf.Clamp(i, 0, nx-1);
    j = Mathf.Clamp(j, 0, ny-1);
    k = Mathf.Clamp(k, 0, nz-1);
    return frame.density[i,j,k];
}

public void ReportProbeData(Vector3 probePosition, string
probeName="Probe") {
    float dens = SampleDensityAt(probePosition);
    probeInfoText.text = $"{probeName} Density = {dens:0.##}";
}
}

// Support classes for JSON parsing (structure depends on your JSON format)
[Serializable]
public class SimulationMeta {
    public int nx, ny, nz;
    public int frameCount;
    public int particleCount;
    public float[] times;
    // Add other fields as per JSON structure
}

[Serializable]
public class LiveFrameMessage {
    public int frameIndex;
    public int totalFrames;
    public int nx, ny, nz;
}

```

```

    // etc...
}

```

Explanation: - We declared public fields to link all required scene objects (we will assign these in Unity Editor via Inspector). - **Start()** sets up UI callbacks (slider and toggles) and calls data loading and WebSocket start if needed. It then initializes the first frame. - **Data Loading:** **LoadSimulationFromFiles** expects a JSON file (parsed via **SimulationMeta** class) and a binary file path. It reads all frames into the **frames** list. We assumed a JSON structure giving dimensions and counts (you should adjust **SimulationMeta** to match your actual JSON schema). We read binary floats accordingly. This uses straightforward nested loops for clarity – it might be slow for very large data, but fine for moderate size. We store density in a 3D float array for easy sampling. - **WebSocket:** We start a background thread to connect and listen (using .NET's **ClientWebSocket**). On receiving messages, we distinguish text vs binary. Text messages are enqueued in **wsJsonQueue**, binary in **wsBinaryQueue**. We lock access because Unity's main thread will dequeue them in Update() loop. We used synchronous **.Result** in the thread (since it's a background thread, this is okay). The thread will block waiting for messages without freezing Unity. - **Update():** Each frame, we empty the queues and process any incoming messages. **ProcessIncomingJson** and **ProcessIncomingBinary** handle assembling frame data from the network. The exact implementation will depend on how your simulation sends data. We provided an example where binary message starts with frame index and particle count, followed by arrays of floats. Adjust this to your protocol. After constructing a **FrameData** from binary, we add it to **frames**. We then update the slider and visuals accordingly (e.g., auto-advance to new frame). - **ApplyFrameData(int frameIndex):** Core method that takes the frame's data and updates all visualization elements: - Calls **UpdateParticleEffect** with the particle positions. - Calls **UpdateDensityVolume** with the density 3D array. - Calls **UpdateSliceTexture** for our one slice (we choose mid-plane for example). - **UpdateParticleEffect:** Creates a **Texture2D** of width = particle count, encodes positions in each pixel (RGB channels). We then assign this texture to the **VisualEffect** and set the particle count, then trigger the "SpawnParticles" event. The VFX Graph then spawns exactly those particles at the given positions. This effectively **GPU-instantiates** points for each position using our graph setup. - **UpdateDensityVolume:** Creates a new **Texture3D** for the density field. We normalize the density to [0,1] within the frame for visibility (you might also use a global max across frames for consistency). Each voxel's color is set to white with alpha = density (so the shader uses alpha as density and tints with **_ColorTint**). We then assign this **Texture3D** to our volume material (**_MainTex**). The shader will immediately start raymarching this data. Because we replace the texture each frame, this is somewhat heavy; a potential optimization is to reuse a **Texture3D** and just update its pixel data (using **SetPixelData** or partial updates), but for clarity we recreate it. - **UpdateSliceTexture:** Extracts a 2D slice from the 3D array at index **sliceIndex**. We compute a simple color mapping (here black-to-cyan) based on normalized density on that slice. Then we assign it to the slice material's main texture. Again, re-created each frame for simplicity. - **UI Event Handlers:** - **OnTimelineSliderChanged:** when user drags the slider, we round to nearest frame and call **ApplyFrameData** for that frame. - We update the **timeLabel** to show either the simulation time or frame index. (In our **SimulationMeta**, we assumed an array of times maybe). - Toggle handlers simply show/hide the corresponding **GameObjects**: enabling/disabling **particleEffect** object, the volume cube, and slice quad. This is a simple way to toggle visualization layers. We also call an **UpdateVisibility()** in **Start** to apply initial toggle states (in case some are off by default). - **Probe Sampling:** We provide **SampleDensityAt(Vector3 worldPos)** to look up the density at a given point in world space in the current frame. This function assumes the volume spans [-0.5,0.5] in each dimension. If your simulation coordinates differ, you'd convert accordingly. We use the position to index into the density array (clamping

inside range). Then `ReportProbeData` forms a string and sets the `probeInfoText`. - We will call `ReportProbeData` from our Probe object's script when a probe is clicked.

A lot is happening in this script, but essentially it bridges **data** -> **visuals** -> **UI**. All heavy computations (texture creation, etc.) are done on the fly when a frame changes. For large grids, this could be slow (perhaps consider doing volume or slice updates less frequently or using compute shaders for performance). But functionally, this works.

Don't forget to attach this script to an object in the scene: - Create an empty GameObject "SimulationController" and add **SimulationController** component. - In Inspector, assign the references: - ParticleEffect → drag the ParticleEffect gameobject. - VolumeMaterial → drag the VolumeMat (the material on the cube). - SliceMaterial → drag SliceMat. - TimelineSlider → drag the Slider. - timeLabel → drag the Text. - particlesToggle, volumeToggle, sliceToggle → drag respective toggles. - probeInfoText → drag the probe info UI Text. - Assign the TextAsset JSON if you put one in Assets (alternatively, you can leave it null and just specify binary path or handle differently). - BinaryDataPath → if using StreamingAssets, you could put your file there and specify `Path.Combine(Application.streamingAssetsPath, "simdata.bin")`. - WebSocketURL → put your simulation's WS address. If you don't have one, you can leave it or disable StartWebSocketListener call if not needed.

Now let's implement the **Probe** object and script that uses the SimulationController's `ReportProbeData`.

Clickable 3D Probes

A probe in this context is a 3D marker in the scene that a user can click to inspect data values at that location (e.g., density). We will create a simple sphere to act as a probe, and a script to handle clicks.

1. Create Probe Object: GameObject → 3D Object → **Sphere**. Name it "Probe1". Scale it down to a small size (e.g., 0.05 uniform) so it looks like a small marker. Position it somewhere within the volume (e.g., at an interesting point you want to sample). For demonstration, place one probe near the center (0,0,0) and maybe another at a different point. You can have multiple probes. - Change the sphere's Material to something bright like pure red or another distinct color, so it's visible. You may even make it emissive (for instance, create a material with Emission enabled and a color, so it glows with bloom). - **Collider:** The Sphere by default comes with a Sphere Collider – keep that, as it will allow it to detect clicks.

2. Create Probe Script: In Scripts, create **Probe.cs**:

```
using UnityEngine;

public class Probe : MonoBehaviour {
    public SimulationController controller; // assign via Inspector

    void OnMouseDown() {
        if (controller != null) {
            controller.ReportProbeData(transform.position, name);
        }
    }
}
```

```
}
}
```

This script simply detects a mouse click on the object (`OnMouseDown` is called by Unity if the object has a collider and is clicked with the mouse) and then calls the `SimulationController`'s `ReportProbeData`, passing its world position and its name. We use the object's name just to label the output.

- Attach this **Probe** script to each probe sphere (select Probe1, Add Component → Probe). In the Inspector, assign the **controller** reference: drag the `SimulationController` object to this field.
- Optionally, duplicate the Probe1 object to make Probe2, etc., for multiple sampling points. The script will automatically use the object's name (so name them "Probe2", "Probe3", etc.) in the output.

Now, at runtime, if you click on a probe sphere, `OnMouseDown` will trigger, and the `SimulationController` will look up the current frame's density at that point, then update the UI text to show something like "Probe1 Density = XYZ". This provides an interactive way to inspect values.

Usage Tip: The probes can be made movable (one could extend the script to allow click-and-drag), or you could spawn new probes at clicked positions. Those are possible extensions. As a simple approach, we placed a few fixed probes. The user can pause or scrub the timeline and click probes to see how density changes at those fixed locations over time.

Final Steps: Testing and Tweaking

With all components in place, **press Play** in the Unity Editor to test the simulation front-end: - If you provided a JSON/Binary data file, it should load on Start. You should see the initial frame's volume rendering (a faint glow in the cube, if density is low) and particles (if any, as points). The timeline slider should reflect the number of frames ¹². Try scrubbing it – as you move the slider, `OnTimelineSliderChanged` will load that frame's data into the visualizations. - Try toggling off "Particles" or "Volume" to ensure the toggles hide/show the respective layers. - If using WebSocket live data, ensure your simulation server is running and sending data. The console will log connection status and frame receipts. New frames should update the visualization automatically (and you can still scrub back with the slider if we stored them). - Click on probe spheres. The `probeInfoText` should update with the density at that point for the current frame. Move the timeline and click again to see how it changes. This imitates how you'd inspect simulation results interactively.

Tweaking Visuals for Sci-Fi Style: - **Bloom Intensity:** Adjust the Bloom post-processing to make the plasma and UI glow more. Be careful: too much bloom can blur details. A bloom threshold around 1.0 and intensity around 0.5–1.5 is often good for neon effects. - **Color Tint:** You can change the `_ColorTint` in the volume material at runtime (or via script based on field type). For example, density could be bluish, another field could be reddish, etc., to differentiate if you had multiple volume fields. - **Particle Appearance:** You can refine the VFX Graph output – for instance, use a small **point mesh** instead of quad for particles if using HDRP (in URP, mesh outputs might not be fully supported, so quads are fine). You can also vary particle color by some attribute (maybe if your data has different particle energies, etc. – then you'd feed that as well via another texture or by encoding color in PositionMap's alpha or a parallel map). - **UI Style:** For a polished look, consider using a **TextMeshPro** text for sharper font rendering (Unity will prompt to import TMP Essentials). TMP also allows adding glows or outline to text which can enhance neon

appearance. For example, an Outline around bright text can make it readable on any background. Additionally, you might use custom sci-fi fonts or iconography for the toggles (the Unity Asset Store or Font sites have free sci-fi fonts/icons). - **Animations:** A subtle UI animation can sell the futuristic feel – e.g., the timeline slider's handle or fill could glow when active, or the panel could slide in. Unity's UI system and animator can handle this, but not necessary for functionality. - **Background and context:** If appropriate, you could add a faint grid or axis lines in the scene (glowing lines) to represent coordinate axes or boundaries – often seen in sci-fi HUDs. This could be done by using LineRenderer or instantiating thin cuboids as axes with emissive material.

Ensure performance is acceptable with your data sizes. If you have a very large grid (say 256^3), the Texture3D creation each frame might be expensive. Monitor Unity's profiler if needed. Some optimization ideas: - Reuse textures instead of creating new every time: create one `Texture3D densityTexture` at start and keep updating its pixel data (Unity allows `texture.SetPixels` or `SetPixelData` with an array, which you can reuse). - For particles, updating a Texture2D each frame is okay for moderate counts (tens of thousands). If you had millions of points, you might need a more efficient method (like feeding a ComputeBuffer to a custom shader for drawing points, or splitting data). - The WebSocket thread approach is a simple way to not block Unity. Just be mindful to properly close the socket when ending the app (we didn't explicitly handle application quit to abort the thread/socket; you might add an `OnDestroy` to clean up the ws).

Finally, **polish the user experience:** - Provide user feedback for interactions: e.g., when clicking a probe, maybe briefly highlight it (you could change its color in `OnMouseDown` or play a sound). - Add units to your labels if needed (e.g., "Density = $5e20 \text{ m}^{-3}$ "). - If the simulation has multiple data fields (magnetic field, temperature, etc.), you could extend the UI with dropdowns or additional toggles to select which field the volume or slice shows. That would involve swapping the data used for the volume rendering and color mapping accordingly. - The interface we built is akin to a scientific visualization tool but with a sci-fi game aesthetic. As a final touch, you could put a **frame around the UI** (maybe a stylized neon border image on the panel) or use **UI images** for buttons instead of plain toggles for a more diegetic look.

Congratulations – you have a working Unity front-end for Dense Plasma Focus simulation data! You can interactively scrub through time ²⁵, view volumetric plasma density, particle distributions, and inspect values at specific points – all in a futuristic-looking interface. This setup can be further expanded (multiple data sets, better rendering effects), but it provides a solid foundation that merges scientific accuracy with game-like visual flair.

Sources:

- Unity Manual: *Using URP in a New Project* – steps for project creation ¹
- Unity Manual: *Visual Effect Graph requirements* – VFX Graph uses Scriptable Render Pipeline and GPU compute ⁷
- Unity Manual: *Installing Visual Effect Graph* – how to add via Package Manager ⁵
- Unity Manual: *Sample 3D Texture in Shader* – example of raymarching a 3D texture for volume rendering ^{23 20}
- Tech-X USim Documentation – example using a timeline slider to scrub simulation time ²⁵
- SitePoint on Sci-Fi UI Design – importance of aesthetics with graphs and minimal text ⁸
- Unity UI Best Practices – use **Scale With Screen Size** for Canvas to handle different resolutions ¹¹

1 Using the Universal Render Pipeline in a new Project | Universal RP | 7.1.8

<https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@7.1/manual/creating-a-new-project-with-urp.html>

2 5 7 Getting started with Visual Effect Graph | Visual Effect Graph | 8.2.0

<https://docs.unity3d.com/Packages/com.unity.visualeffectgraph@8.2/manual/GettingStarted.html>

3 4 Visual Effect Graph does not work with URP - Unity Discussions

<https://discussions.unity.com/t/visual-effect-graph-does-not-work-with-urp/244847>

6 Introduction to the VFX Graph in Unity

<https://unity.com/how-to/introduction-vfx-graph-unity>

8 14 Top Sci-Fi Designs to Inspire Your Next Interface - SitePoint

<https://www.sitepoint.com/14-top-sci-fi-designs-to-inspire-your-next-interface/>

9 10 11 Unity UI System - Best Practices - DEV Community

<https://dev.to/marbleit/unity-ui-system-best-practices-2o24>

12 25 Dense Plasma Focus (densePlasmaFocus.pre) — USim 3.0.1 documentation

https://txcorp.com/images/docs/usim/latest/quick_start/USimHEDP/densePlasmaFocus/densePlasmaFocus.html

13 14 15 16 Set Attribute from Map | Visual Effect Graph | 10.2.2

<https://docs.unity3d.com/Packages/com.unity.visualeffectgraph@10.2/manual/Block-SetAttributeFromMap.html>

17 18 19 20 21 22 23 24 Unity - Manual: Sample a 3D texture in a shader

<https://docs.unity3d.com/6000.0/Documentation/Manual/class-Texture3D-use-in-shader.html>